

Engineering-First C# Architecture

Overview

This is a reference architecture for building engineering calculation tools in C#. It prioritizes **clarity, traceability, and long-term maintainability** over architectural complexity.

Core principle: Write code you can understand 6 months later without a whiteboard.

Design Philosophy

This architecture follows an **Explicit Functional** core approach:

- Domain logic expressed as **pure, explicit functions**
 - no local functions, no lambda expressions outside of LINQ where they're standard. If a helper is needed, it gets its own private static method with a proper name and signature.
- Data represented by **simple, strongly typed records**
- I/O, UI, and persistence kept at the **edges**
- No unnecessary frameworks, ORMs, or architectural ceremony

Goal: Keep the path from *physical model* → *equations* → *code* → *results* as short and visible as possible.

When to Use This Architecture

✓ Good fit for:

- Engineering calculations and simulations
- Scientific computing tools
- Batch processing systems
- Tools requiring verification and traceability
- Small to medium teams
- Long-lived projects that must remain maintainable

✗ Poor fit for:

- Large-scale enterprise applications with many teams
 - Real-time interactive systems with complex state management
 - Applications where UI is the primary complexity
-

Recommended Project Structure

ProjectName/

|

```
|— Core/
|   |— Models/      # Data-only types (records/structs)
|   |— Calculations/ # Pure calculation functions
|   |— IO/          # CSV, XML, file operations
|
|— Console/ # Command-line interface
|— GUI/     # Desktop UI
```

Structural Rules

- **Core has NO UI or database references**
 - **Models contain NO logic** (data only)
 - **Calculations contain NO I/O** (pure functions)
 - **UI projects only orchestrate** (thin shells)
-

Core Principles

1. Treat C# as Structured, Typed, Procedural Language

csharp

```
// This is GOOD engineering code
```

```
double BendingStress(double moment, double inertia)
    => moment / inertia;
```

Use:

- Functions with clear names
- Explicit data structures
- Minimal abstractions

Avoid:

- Clever abstractions
- "Extensibility for the future"
- Over-engineering

2. Use Records for Data

csharp

```
record Section(
    double Area,
    double Inertia,
    double SectionModulus
);
```

Why records?

- Immutable by default
- Value semantics
- Self-documenting
- No boilerplate

3. Classes Only When Necessary

Use classes **only** if:

- It owns resources (file, database connection)
- It has internal state that evolves
- Lifetime management matters

Otherwise, prefer records or plain functions.

4. Pattern Matching for Decision Logic

csharp

```
double AllowableStress(Section s) => s switch
{
    { Area: < 0.01 } => 0,
    { Inertia: < 1e-6 } => 0,
    _ => 250e6
};
```

This replaces:

- Long if-else chains
- Complex OOP hierarchies
- Obscure polymorphism

5. Interfaces Only at Real Boundaries

Use interfaces when:

- Talking to external systems (database, API, files)
- You truly need to swap implementations

Don't use interfaces for:

- Internal calculations
- Data models
- "Just in case" extensibility

If you have one implementation, you don't need an interface.

6. Avoid Inheritance

Instead of polymorphic hierarchies, prefer:

- Explicit functions per type

- Enums + pattern matching
- Separate, clear implementations

Explicit beats polymorphic cleverness for engineering.

What This Architecture Is NOT

This is **not**:

- MVC (Model-View-Controller)
- MVVM (Model-View-ViewModel)
- Clean Architecture
- Domain-Driven Design (DDD)
- Hexagonal Architecture

These are **organizational architectures** for complex enterprise systems.

This is a **computational architecture** for engineering tools.

Example: Vessel Maneuvering Calculations

csharp

```
// Models/Vessel.cs
record Vessel(double Mass, double Length);
record NomotoParameters(double T, double K);

// Calculations/NomotoModel.cs
static class NomotoModel
{
    public static double YawRate(
        double rudderAngle,
        double time,
        NomotoParameters p)
    {
        return p.K * rudderAngle * (1 - Math.Exp(-time / p.T));
    }
}

// Console/ConsoleUI.cs
var vessel = new Vessel(Mass: 1200, Length: 24.0);
var nomoto = new NomotoParameters(T: 15.0, K: 0.12);
var yawRate = NomotoModel.YawRate(
    rudderAngle: 10.0 * Math.PI / 180.0,
    time: 100.0,
    nomoto
```

```
);  
Console.WriteLine($"Yaw rate: {yawRate:F4} rad/s");
```

NO Testing

Why no test framework?

- Deterministic verification
 - No hidden test runners
 - Code is self-documenting
 - Output reads like an engineering validation report
-

Persistence Strategy

For small applications:

- Use CSV for tabular data
- Use TXT for file saving/loading
- Use XML for configuration
- Raw SQL with Dapper (if database needed)

Avoid:

- JSON
- Entity Framework (unless schema is large and complex)
- ORMs that hide SQL
- Over-abstraction

Principle: Abstraction is only valuable when it hides pain, not when it hides understanding.

How to Grow Safely

Stage 1: Single Executable

Everything in one project. Simple and clear.

Stage 2: Extract Core Library

Move calculations to separate project when logic stabilizes.

/Core ← Pure calculations + models

/Console ← Command-line interface

Stage 3: Add Multiple Front-Ends

/Core ← Unchanged

/Console

/ImGui

/Wpf

/WebApi

Key insight: UI layers are plug-ins. Core is invariant.

What NOT to Add (Even as It Grows)

- Dependency Injection containers
 - Repository pattern
 - Abstract base class hierarchies
 - Reflection-heavy frameworks
 - Over-generalization
-

Decision Guidelines

Before adding complexity, ask:

1. **Does this make the calculation clearer?**
2. **Can I explain this to myself in 6 months?**
3. **Is this hiding a formula or exposing it?**
4. **Does this reduce or increase verification effort?**
5. **Is this solving a real problem today?**

If any answer is "no" → Don't add it.

Modern C# Features to Use

Embrace these freely:

- Strong typing
- Nullable reference types
- Records and structs
- Pattern matching
- `Span<T>` (for performance)
- Excellent IDE refactoring

Ignore architectural trends until genuinely needed.

Why This Ages Well

This approach:

- Scales down perfectly
- Scales up reasonably
- Survives UI rewrites
- Survives developer turnover
- Is testable by inspection

- Maps directly to textbooks
- Respects how engineers think

Engineering software is computation-first, not interaction-first.

Alternative Names for This Architecture

Depending on context, you might call this:

- **Explicit Functional Core** (most accurate)
 - **Computational Core Architecture**
 - **Data-First Engineering Architecture**
 - **Engineering-Driven Programming**
 - **Functional Core, Imperative Shell**
-

Intended Audience

- Engineers
 - Researchers
 - Technical analysts
 - Anyone who values explicit, readable computational code
 - Teams that prioritize correctness and longevity over trendy patterns
-

Summary

A modular, data-first reference architecture for engineering applications, emphasizing explicit domain models, minimal abstraction, and optional UI layers.

This architecture trades fashionable complexity for long-term clarity—exactly what engineering calculations require.

License

CC-BY-SA-4.0 (Attribution-ShareAlike 4.0 International) by Creative Commons Corporation
(creativecommons.org)